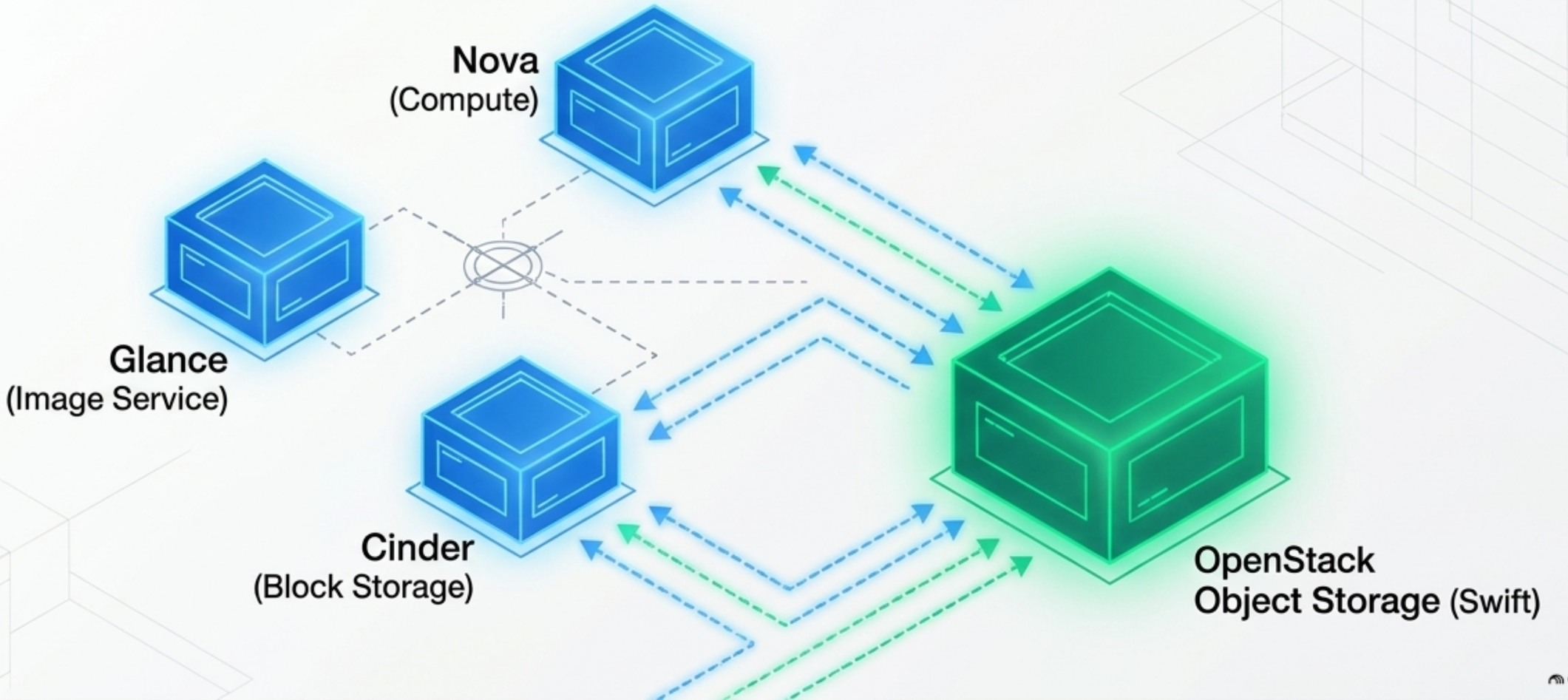
เจาะลึกสถาปัตยกรรม OpenStack Object Storage (Swift)

โครงสร้างการจัดเก็บข้อมูล กลไกการทำ Replication และการทำงานร่วมกับระบบนิเวศคลาวด์

Gemini Notebook

ตำแหน่งของ Swift ในระบบนิเวศ OpenStack

OpenStack ใช้สถาปัตยกรรมแบบกระจายศูนย์ (Shared-nothing architecture) ทำงานร่วมกันผ่าน Message Queue (AMQP) และ HTTP API โดย Swift ทำหน้าที่เป็นระบบจัดการพื้นที่เก็บข้อมูลแบบอ็อบเจกต์ (Object Store)



Gemini Notebook

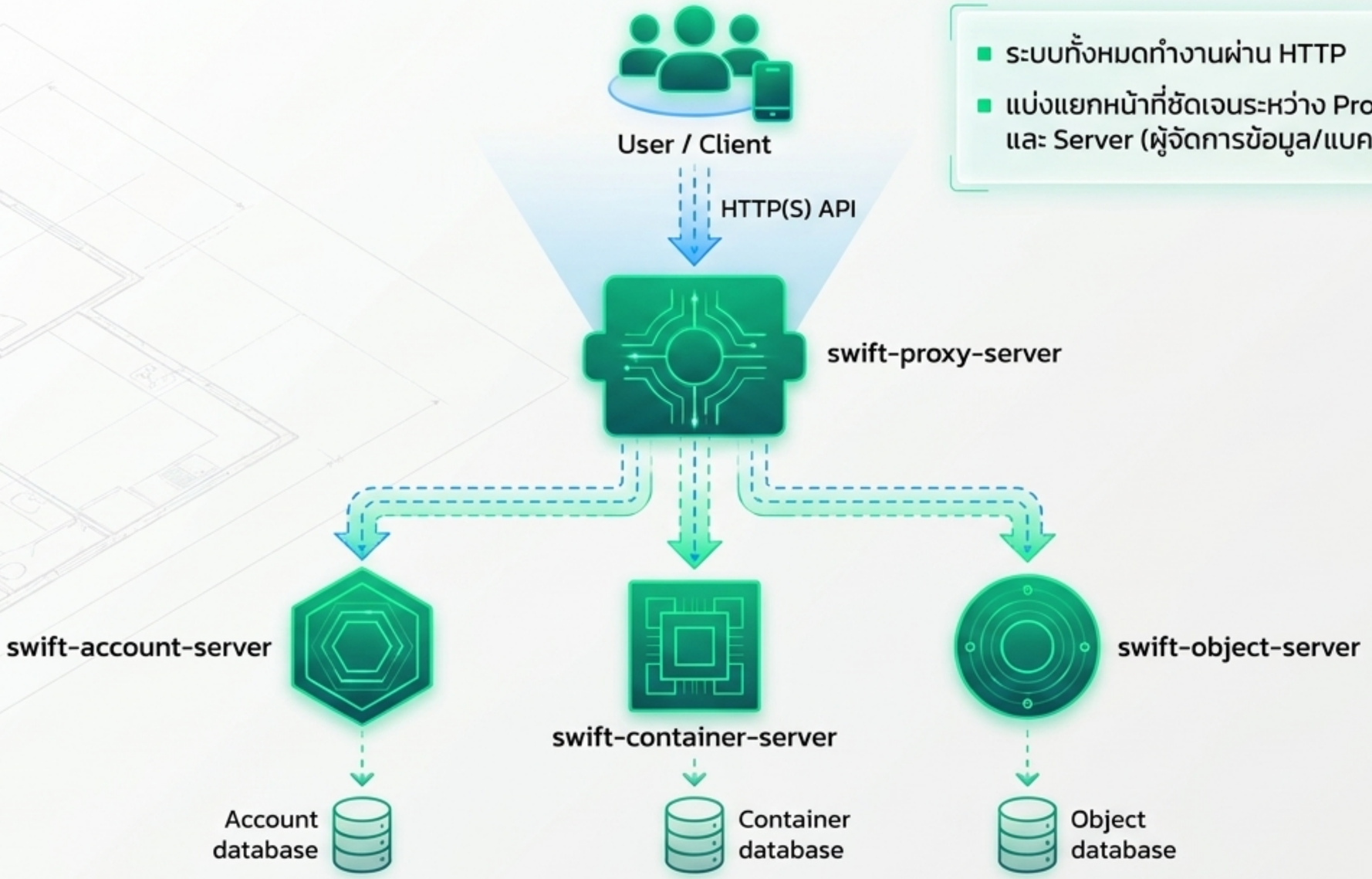
ทำไมต้อง Object Storage? (วิเคราะห์เปรียบเทียบ Storage 3 ประเภท)

Ephemeral Storage (Nova)	Block Storage (Cinder)	Object Storage (Swift)
ลักษณะ: ผูกติดกับอายุขัยของ Instance (Root disk)	**ลักษณะ:** Persistent Volume คงอยู่ถาวร	**ลักษณะ:** เก็บข้อมูลแบบอ็อบเจกต์ผ่าน HTTP/API
การคงอยู่: ข้อมูลหายไปเมื่อ Instance ถูกลบ	**การคงอยู่:** แยกอิสระจาก Instance และคงอยู่แม้ Instance ปิดตัวลง แต่อ่าน/เขียนได้จาก Instance เดียวในเวลาหนึ่ง	**การคงอยู่:** ขยายสเกลได้ไม่จำกัด รองรับการเข้าถึงพร้อมกันจากหลายจุด
Use Case: เก็บระบบปฏิบัติการชั่วคราว (เช่น Ext4)	**Use Case:** ระบบฐานข้อมูล, ข้อมูลที่ต้องการประสิทธิภาพ I/O สูง	**Use Case:** การเก็บไฟล์ขนาดใหญ่, Backup, และจัดเก็บ Image ของระบบ

Gemini Notebook

สถาปัตยกรรมภายในของ OpenStack Swift

- ระบบทั้งหมดทำงานผ่าน HTTP
- แบ่งแยกหน้าที่ชัดเจนระหว่าง Proxy (ผู้จัดการจราจร) และ Server (ผู้จัดการข้อมูล/แบคเอนด์)



Gemini Notebook

Proxy Server: สมองสั่งการและคนเฝ้าประตู

HTTP Requests

API Gateway: รับและตอบกลับ HTTP Requests (PUT, GET, DELETE) จากไคลเอนต์

Authentication: ยืนยันตัวตนกับ Keystone ก่อนอนุญาตให้เข้าถึงข้อมูล



AMQP/HTTP Mapping

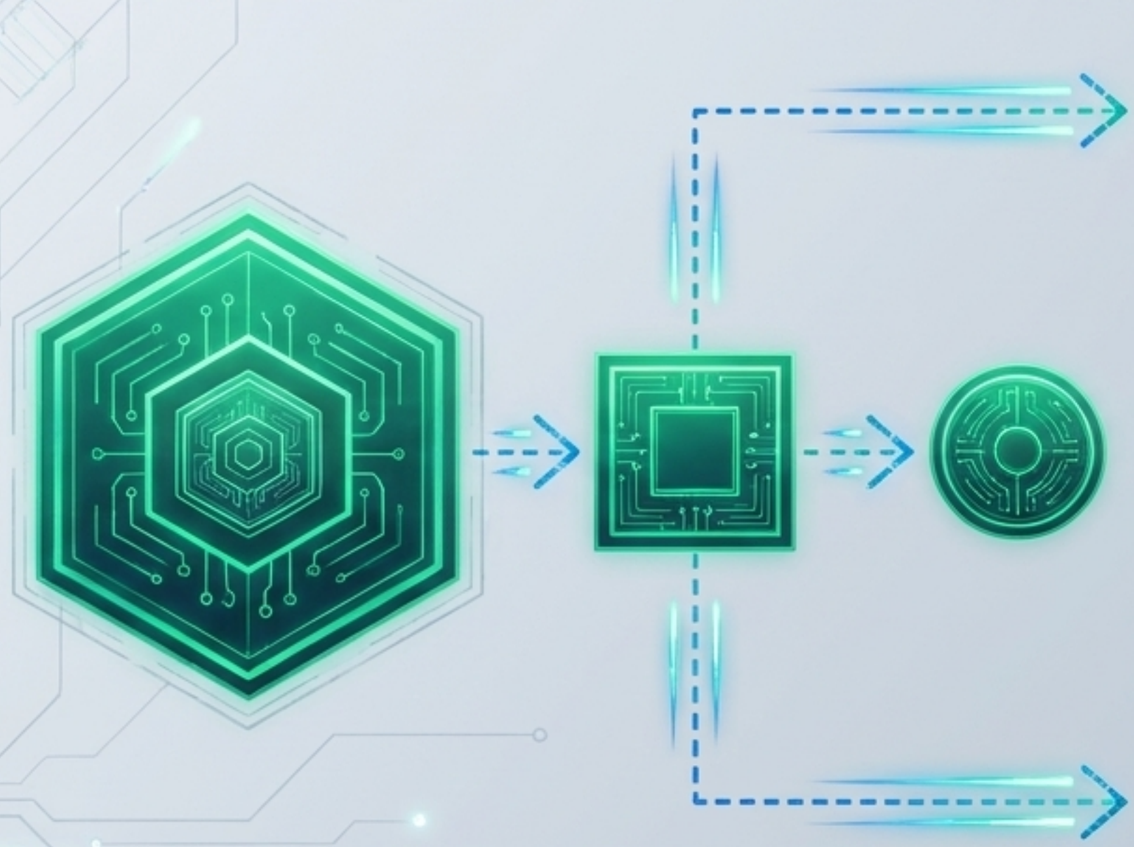


Data Routing (The Ring): ใช้กลไก ‘Ring’ เพื่อทำแผนที่ (Mapping) ว่าข้อมูลชิ้นใดควรถูกเก็บหรือดึงมาจากโหนด (Node) ไหนในคลัสเตอร์

* Proxy Server ไม่ได้เก็บฐานข้อมูลหรือไฟล์จริง (Stateless)

Gemini Notebook

โครงสร้างลำดับชั้นของข้อมูล (Account, Container, Object)



Account (Tenant): ระดับบนสุด มักจะผูกกับโปรเจกต์ (Project) หรือผู้ใช้งาน (Tenant) เพื่อจัดการสิทธิ์และโควต้า

Container (Bucket): เปรียบเสมือนโฟลเดอร์สำหรับจัดกลุ่มอ็อบเจกต์ (ไม่สามารถสร้าง Container ซ้อน Container ได้)

Object (File + Metadata): ข้อมูลไบนารีที่แท้จริง (เช่น รูปภาพ, วิดีโอ, ไฟล์ log) พร้อมกับ Metadata ที่อธิบายตัวมันเอง

Gemini Notebook

Account & Container Servers: ผู้จัดการ Metadata





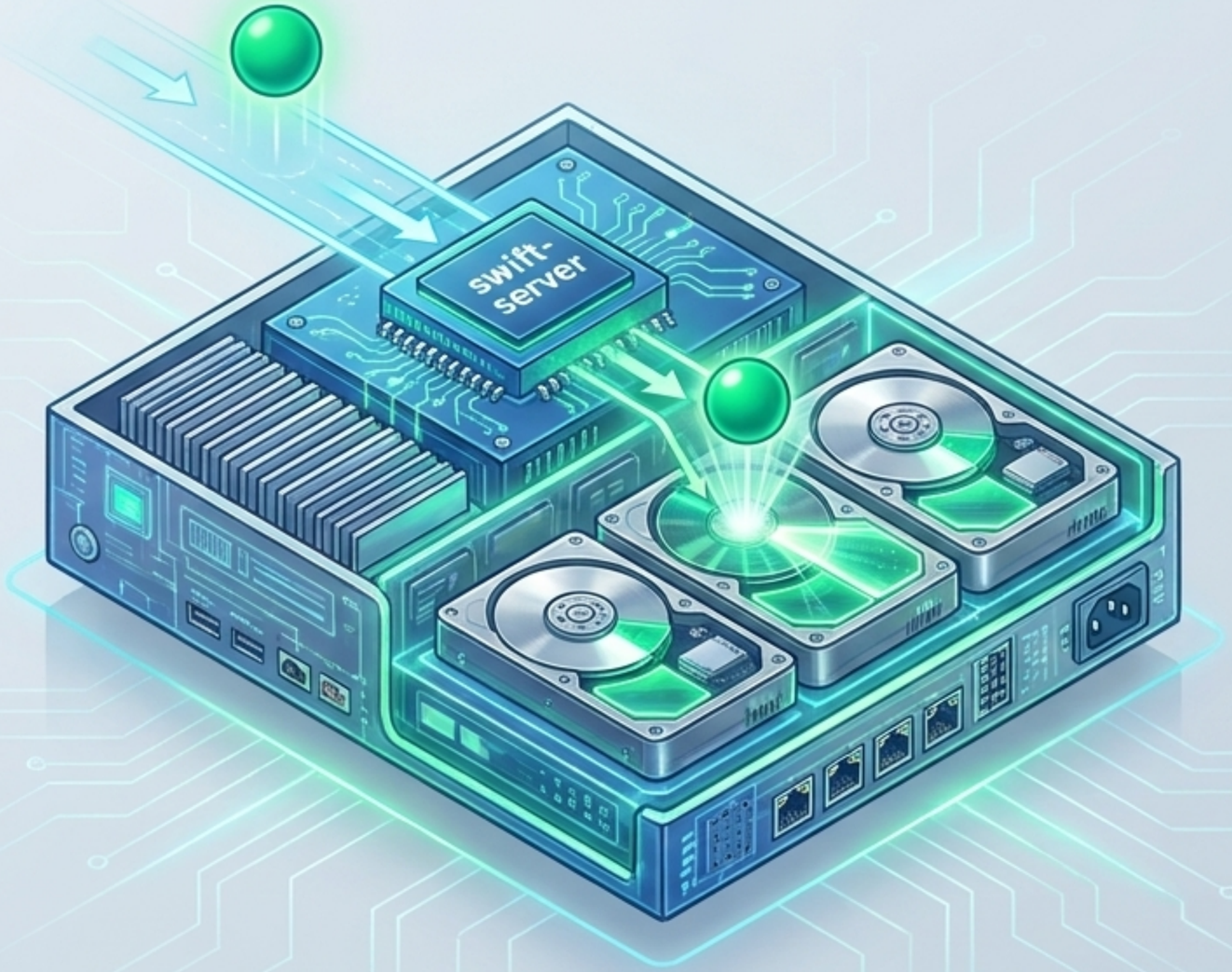
swift-account-server: จัดการ SQLite Database ทำหน้าที่บันทึกรายชื่อของ Containers ทั้งหมดที่อยู่ใน Account นั้น

swift-container-server: จัดการ SQLite Database ทำหน้าที่บันทึกรายชื่อของ Objects ทั้งหมดที่อยู่ใน Container นั้น

เซิร์ฟเวอร์ทั้งสองนี้ไม่ได้จัดเก็บเนื้อหาไฟล์ (Payload) แต่เก็บเฉพาะ ‘รายการชี้เป้า’ เพื่อให้การค้นหามีความรวดเร็วและตรวจสอบความถูกต้องได้ทันที

Gemini Notebook

The Object Server: แหล่งจัดเก็บข้อมูลทางกายภาพ



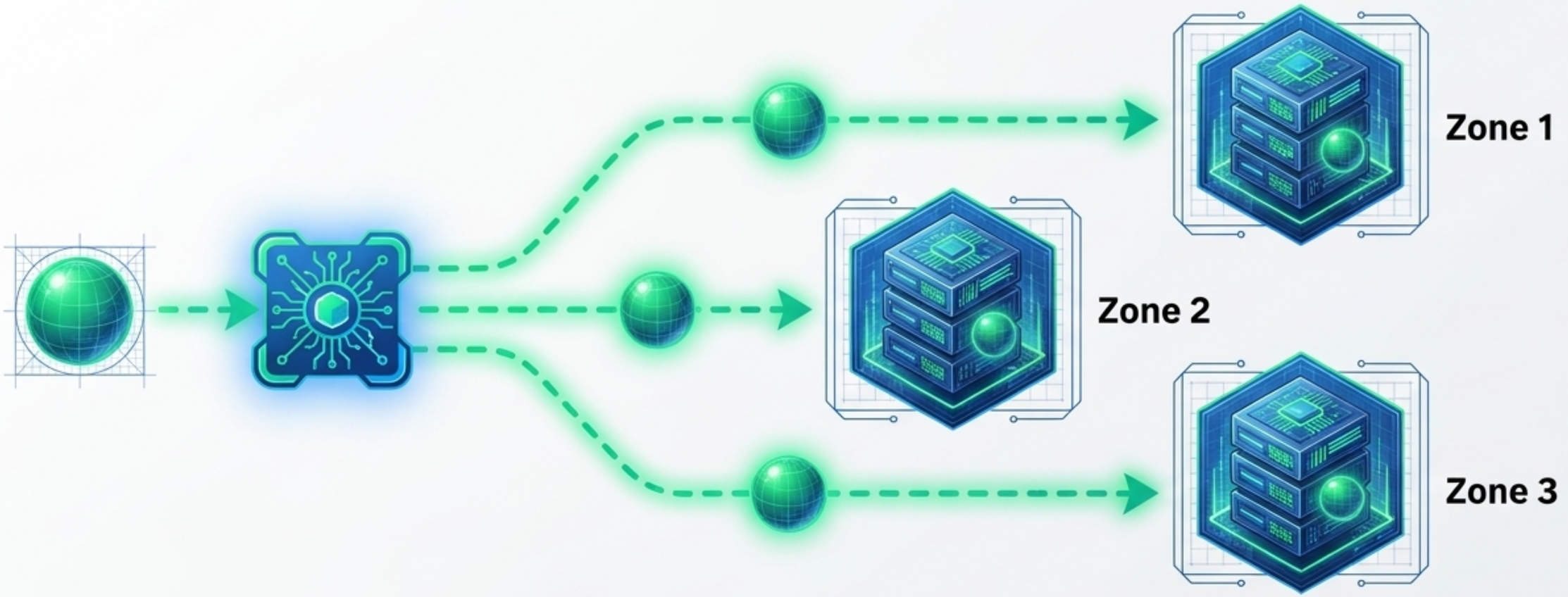
swift-object-server: รับหน้าที่เขียนและอ่านอ็อมูลไบนารี (Binary Data) ลงบนพื้นที่เก็บข้อมูลของโหนด (Disks) จริงๆ

Path Mapping: ข้อมูลจะถูกเก็บเป็นไฟล์บนระบบไฟล์มาตรฐาน (เช่น XFS หรือ Ext4) โดยใช้ค่า Hash เป็นชื่อไฟล์

Metadata Attached: ข้อมูล Metadata ของอ็อบเจกต์จะถูกฝังไว้ใน Extended Attributes (xattrs) ของไฟล์นั้นๆ เพื่อให้ Object สามารถอธิบายตัวเองได้โดยอิสระ

Gemini Notebook

Data Durability: กลไกการคัดลอกเพื่อความทนทาน (Replication)



Default 3x Replication

ทุกๆ อ็อบเจกต์ที่อัปโหลดเข้ามา
จะถูก Proxy Server ทำสำเนาโดยอัตโนมัติ
(ค่าเริ่มต้นคือ 3 ชุด)

Zone Separation

สำเนาแต่ละชุดจะถูกกระจายเก็บใน Node
หรือ Availability Zone (AZ)
ที่แตกต่างกัน

Self-Healing

หากฮาร์ดแวร์โหนดใดโหนดหนึ่งล่ม
ระบบจะดึงสำเนาจากโซนอื่นมาใช้งานแทนทันที
และทำสำเนาใหม่ทดแทนส่วนที่หายไปอัตโนมัติ

Gemini Notebook

Ecosystem Synergy: การทำงานร่วมกันระหว่าง Swift และ Glance

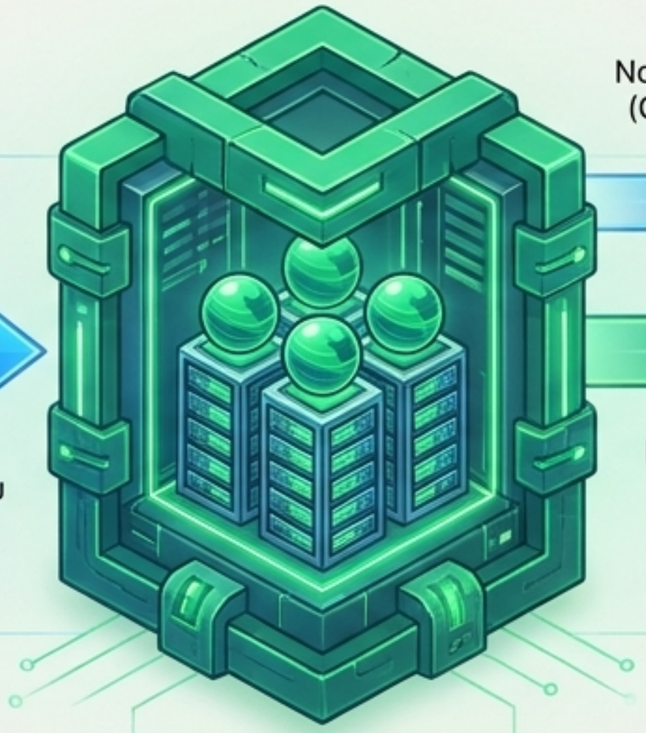
1. Glance (Image Service)



รับหน้าที่จัดการ Image ของระบบปฏิบัติการ (เช่น Ubuntu, Windows)

Glance API
ผลัก Image ไปเก็บ
(Push Image)

2. Storage Backend (Swift)

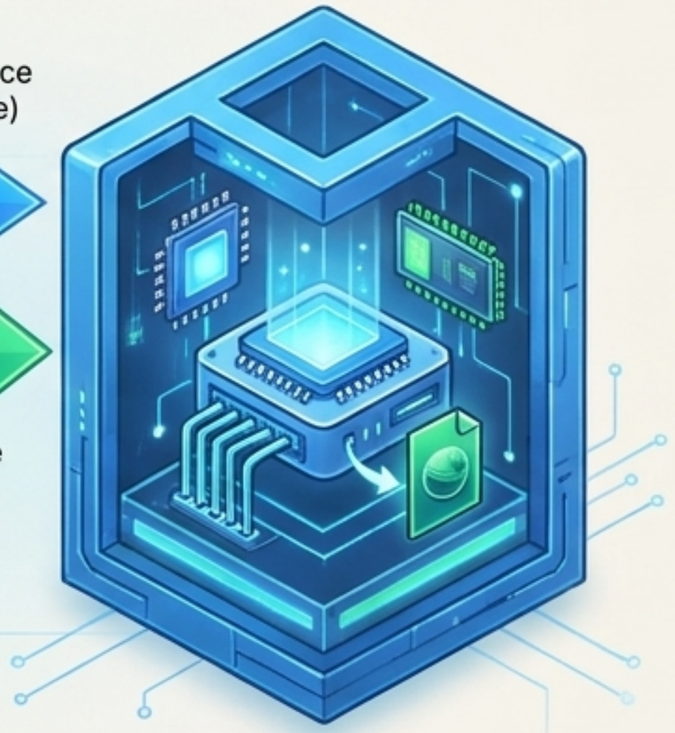


Glance API จะผลักไฟล์ Image ขนาดใหญ่มาเก็บไว้ใน Swift เนื่องจากมีความทนทานสูงและรองรับไฟล์ใหญ่ได้ดี

Nova สร้าง Instance
(Create Instance)

Nova ดึง Image
ผ่าน Glance
(Pull Image)

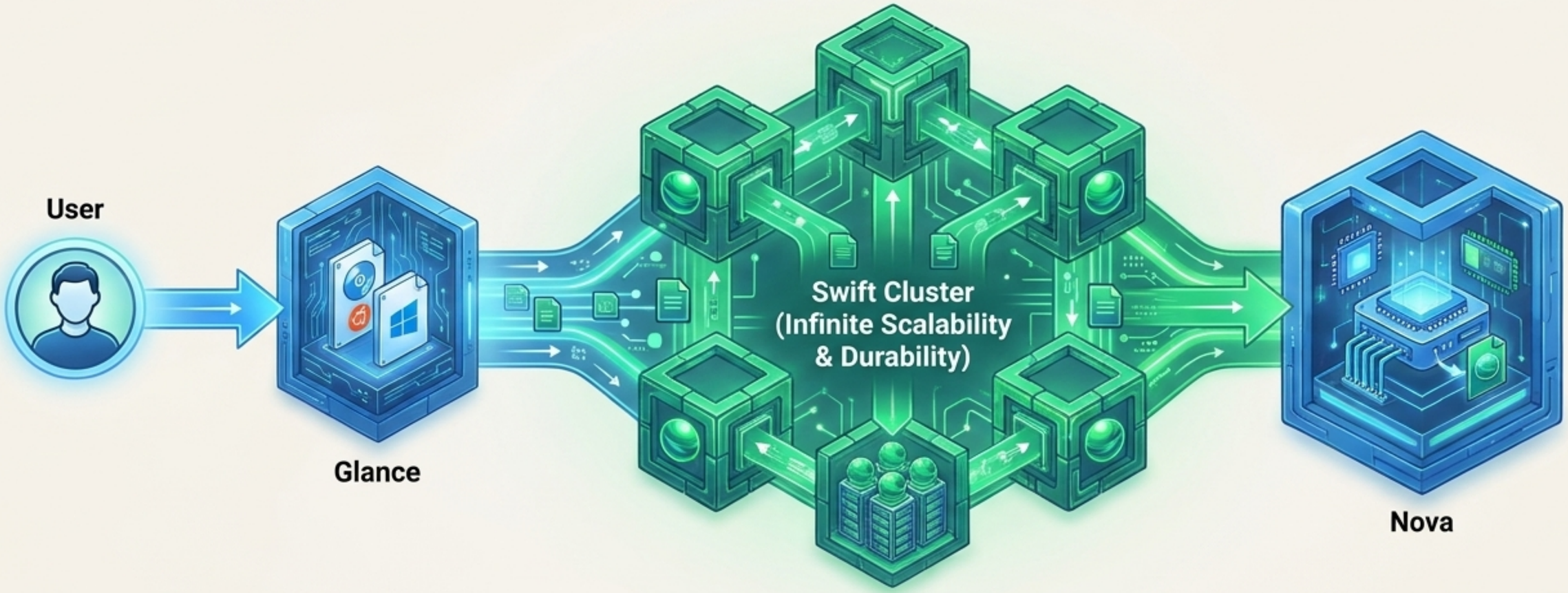
3. Nova (Compute)



เมื่อผู้ใช้ต้องการสร้าง Instance ใหม่ Nova จะดึง Image ผ่าน Glance ซึ่งไปดึงข้อมูลจริงมาจากสถาปัตยกรรมอันแข็งแกร่งของ Swift อีกทอดหนึ่ง

Gemini Notebook

บทสรุป: สถาปัตยกรรมที่ออกแบบมาเพื่อการขยายตัวอย่างไร้ขีดจำกัด



Infinite Scalability:
แยกส่วน Proxy และ Server ออกจากกัน ทำให้ขยายระบบได้ตามความต้องการ

Extreme Durability:
มั่นใจได้ด้วยกลไก Self-healing และ 3x Replication

Seamless Integration:
เป็นรากฐานข้อมูลที่ไร้รอยต่อสำหรับบริการอื่นๆ ในระบบนิเวศ OpenStack

Gemini Notebook